ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
Мэдээлэл холбооны технологийн сургууль

БИЕ ДААЛТЫН АЖЛЫН ТАЙЛАН

Үйлдлийн систем (F.CSM302)

2026-2027 оны хичээлийн жилийн намар

Бие даалтын ажлын нэр: The Dining-Philosophers Problem — Бие даалт 1 (хуваарилалтын код II-4-11)

Хичээл заасан багш: Ганбат

Лабораторийн ажил гүйцэтгэсэн: Оюутан: Г. Сэлэнгэ (B242270106)

Оюутан: Ч. Гантогтох (B242270005)

Оюутан: Э. Энхсаруул (B232270145)

Оюутан: Э. Төгөлдөр (B241940019)

Улаанбаатар хот
2026 он

ТОЙМ

Dining-Philosophers бол олон thread зэрэгцээ ажиллахад хуваалцсан хязгаарлагдмал нөөцийг (энд: fork) хэрхэн deadlock-гүйгээр хуваарилах тухай сурах бичгийн сонгодог асуудал гэж тодорхойлогдоно. Энэ ажлын зорилго нь асуудлыг C/pthread дээр хэрэгжүүлж, хэрэглэгчийн тохиргоогоор дахин туршигдах боломжтой, бодит ажиллагаатай холбогдсон интерактив state-diagram UI-аар шалгаж харуулахад оршино. Арга зүйн хувьд хоёр хувилбар хэрэгжүүлэгдсэн: сурах бичгийн monitor шийдэл (mutex болон condition variable ашигласан, хөрш хоёрын төлвийг шалгадаг) ба зориудаар deadlock-той хийгдсэн naive (зүүн, дараа нь баруун fork авдаг) шийдэл. Туршилтын үр дүнд monitor хувилбар шалгасан бүх тохиргоонд (n=2, n=5, n=64) deadlock, mutual-exclusion зөрчил илрээгүй; starvation нь сурах бичгийн дагуу онолын хувьд боломжтой хэдий ч манай туршилтуудад ажиглагдаагүй. Naive хувилбар нь бодох хугацааг тогтмол (think=0, random бус) болгож, идэх хугацааг random (300-600мс) орхиод, тохирсон эерэг grab-delay-тэй (бүх philosopher зэрэг зүүн fork-оо авахад хангалттай цонх) үед 3 удаагийн давтан туршилтад (seed=1,2,3) тогтвортой deadlock-д орсон. n=5, 4 секундын нэг ажиллагаанд philosopher бүр дунджаар ойролцоогоор 59-60 удаа идэж, дундаж хүлээх хугацаа ≈15мс байсан нь ажиглагдсан (ажиллагаа бүрт OS-ийн thread scheduling-ийн улмаас яг тоо бага зэрэг хувьсдаг). ThreadSanitizer болон valgrind helgrind/leak-check (monitor, naive хоёуланд нь) тухайн ажиллагаанд race condition, санах ойн алдагдал илрүүлээгүй бөгөөд бүх автомат шалгалт (16/16) амжилттай дууссан байна. Эдгээр ажиглалт нь monitor-ийн сурах бичигт заасан онолын шинж чанар (deadlock-гүй, mutual exclusion хангагдсан) практикт тохирч байгааг харуулж байна.

ГАРЧИГ

Тойм..... ii

Зургийн жагсаалт..... iv

Хүснэгтийн жагсаалт..... v

1. Удиртгал..... 1

2. Сурах бичгийн шаардлага..... 1

3. Онолын үндэс..... 1

4. Системийн зохиомж..... 2

5. Хэрэгжилт..... 3

6. Туршилт ба UI дүрслэл..... 3

7. Туршилтын арга..... 4

8. Үр дүн..... 4

9. Хэлэлцүүлэг..... 6

10. Дүгнэлт..... 7

Ашигласан материал..... 8

Хавсралт..... 8

(Дээрх хуудасны дугаарууд энэ бариmтын эцсийн render хийсэн хувилбар дээр үндэслэсэн статик жагсаалт тул Word-д нээгдэх үед автоматаар дахин тооцоологдохгүй. Хэрэв та агуулгад томоохон өөрчлөлт оруулсан бол зөвхөн энэ хүснэгтийн тоонуудыг гараар шалгаж засаарай.)

ЗУРГИЙН ЖАГСААЛТ

Зураг	Гарчиг	Хуудас
Зураг 8.1	Monitor алгоритм, n=5, ажиллагаа дууссаны дараах state-diagram UI ба статистикийн хүснэгт	5
Зураг 8.2	Naive алгоритм дахь детерминист deadlock (бүх philosopher hungry, улаан хүрээтэй)	6

ХҮСНЭГТИЙН ЖАГСААЛТ

Хүснэгт	Гарчиг	Хуудас
Хүснэгт 7.1	Туршилтын тохиргоонууд	4
Хүснэгт 8.1	Monitor алгоритмын тоон үр дүн (n=5, think/eat=10-40мс, 4с, seed=1)	4

1. УДИРТГАЛ

Энэхүү бие даалтын ажлын зорилго нь олон thread зэрэгцээ ажиллах үед хуваалцсан нөөцийн синхрончлолыг хэрхэн зөв зохион байгуулахыг Dining-Philosophers сонгодог асуудлын жишээн дээр практикт судлахад оршив [3]. Ач холбогдлын хувьд энэ бол үйлдлийн системийн синхрончлолын сурах бичигт (Silberschatz et al., 7-р бүлэг) байнга ишлэгддэг сонгодог жишээ бөгөөд энд үзүүлсэн monitor-д суурилсан шийдэл нь бодит үйлдлийн систем, мэдээллийн сангийн түгжээний менежментэд өргөн хэрэглэгддэг зарчмуудтай шууд холбоотой. Ажлын хамрах хүрээ нь: (1) асуудлыг C/pthread дээр deadlock-гүй (monitor) болон зориудаар deadlock-той (naive) хоёр хувилбараар хэрэгжүүлэх, (2) хэрэглэгчийн тохиргоогоор (philosopher-ийн тоо, хугацааны муж, seed) дахин туршигдах боломжтой болгох, (3) бодит ажиллагаатай холбогдсон интерактив state-diagram UI-аар үр дүнг дүрслэх, (4) бүх үр дүнг автомат тест, ThreadSanitizer, valgrind-аар бодитоор баталгаажуулахад оршино. Хуваарилалтын код II-4-11.

2. СУРАХ БИЧГИЙН ШААРДЛАГА

Operating System Concepts сурах бичгийн 7-р бүлэг "Synchronization Examples"-ийн төгсгөлд байрлах "Programming Projects — Project 3: The Dining-Philosophers Problem" [1, х. P-39] нь POSIX хувилбарт тодорхой шаардлага тавьдаг: яг таван philosopher үүсгэж, philosopher бүрийг тусдаа thread-ээр ажиллуулах; philosopher бүр thinking ба eating хооронд 1-ээс 3 секундын хооронд санамсаргүй хугацаагаар sleep хийх; philosopher хоол идэхийг хүсэхдээ pickup_forks(int philosopher_number) функцийг, идэж дуусаад return_forks(int philosopher_number) функцийг дуудах; шийдэл нь 7.1.3-р хэсэгт ("The Dining-Philosophers Problem", х. 293-294, Figure 7.7) өгөгдсөн monitor-д суурилсан алгоритм (state[] массив, condition self[5], pickup(i)/putdown(i)/test(i) функцууд) дээр үндэслэх ёстой. Хэрэгжилтийг эх кодтой шууд тулгахад (src/dp.c) функцийн нэр (pickup_forks, return_forks), анхдагч philosopher-ийн тоо (n=5), бодох/идэх хугацааны анхдагч муж (1000-3000мс) болон monitor-ын state/test/pickup/putdown логик бүгд сурах бичгийн тодорхойлолттой яг тохирч байгааг баталгаажуулсан. Үүн дээр нэмж, philosopher-ийн тоог 2-оос 64 хүртэл динамик өөрчлөх боломжтой болгож (сурах бичгийн тогтмол "тав" гэсэн тооноос давсан уян хэрэгжилт), мөн зориудаар deadlock-той naive хувилбарыг нэмж харьцуулалт хийх боломжийг бий болгосон.

3. ОНОЛЫН ҮНДЭС

Mutual exclusion гэдэг нь нэгэн зэрэг зөвхөн нэг thread хуваалцсан нөөцийг (энд: нэг fork) эзэмшиж болохыг тодорхойлдог зарчим юм [1]. Deadlock (мухардал) нь дараах дөрвөн нөхцөл нэгэн зэрэг биелэх үед үүснэ: (1) mutual exclusion — нөөц хуваалцагдахгүй, (2) hold-and-wait — thread нэг нөөцийг барьсан хэвээр нөгөөг хүлээдэг, (3) no preemption — нөөцийг хүчээр авч

чадахгүй, (4) circular wait — thread-үүд тойрог хэлбэрээр бие биенээ хүлээдэг [1]. Naive хэрэгжилтэд дээрх дөрвөн нөхцөл бүрэн биелдэг тул deadlock үүснэ (доор 5-р хэсэгт яг хэрхэн биелж байгааг тайлбарлав). Харин monitor шийдэлд philosopher бүр хоёр fork-оо нэг атомик алхмаар (test() функцээр) авдаг тул hold-and-wait нөхцөл арилж, deadlock онолын хувьд үүсэх боломжгүй болдог [1]. Анхаарах нь: энэ нь зөвхөн deadlock-гүй байхыг баталгаажуулдаг бөгөөд starvation (нэг philosopher тодорхойгүй удаан хугацаагаар EATING болохгүй байх)-ээс бүрэн хамгаалдаггүйг сурах бичигт тэмдэглэсэн байдаг [1] — манай туршилтуудад starvation ажиглагдаагүй ч энэ нь тестийн хугацаа хязгаарлагдмал байсантай холбоотой байж болно. Хэрэгжилтэд ашигласан pthread_mutex_t ба pthread_cond_t API-ийн семантик нь POSIX Threads стандартаар тодорхойлогддог [2].

Өгөгдлийн бүтцийн хувьд monitor хувилбарт philosopher бүрийн төлвийг (THINKING/HUNGRY/EATING) хадгалах int массив, нийтлэг нэг pthread_mutex_t, болон philosopher тус бүрт зориулсан pthread_cond_t массив ашиглагдсан. Naive хувилбарт харин fork тус бүрд тусдаа pthread_mutex_t олгож, philosopher зүүн, дараа нь баруун fork-оо дараалан trylock-оор (блоклодоггүй, амжилтгүй бол шууд буцдаг) авахыг оролддог. Гагцхүү энд trylock нэг удаа дуудагдаад зогсохгүй, амжилттай болтол 1мс тутамд давтан дуудагддаг тул үр дүнд нь fork-оо хараахан аваагүй үед busy-wait хэлбэртэй блоклолт болдог; харин аль нэг fork-оо нэгэнт авсан бол нөгөөг нь оролдож байх хугацаанд эзэмшсэн fork-оо суллахгүй тул энэ нь яг hold-and-wait нөхцлийг (fork-оо барьсан хэвээр нөгөөг нь хүлээх) бий болгодог. Хоёр хувилбар хоёулаа event-ээ JSON Lines (NDJSON) бүтцээр stdout руу бичдэг тул UI болон тестийн хэрэгсэл stream хэлбэрээр уншиж боловсруулдаг.

4. СИСТЕМИЙН ЗОХИОМЖ

Систем гурван давхаргаас бүрдэнэ: (1) src/dp.c — n philosopher thread, хоёр алгоритм (philo_monitor, philo_naive), event-ийг JSON Lines (NDJSON) хэлбэрээр stdout руу бичдэг цөм логик; (2) ui/server.py — зөвхөн 127.0.0.1-д сонсдог, dp процессыг subprocess-ээр ажиллуулж гаралтыг HTTP streaming-ээр дамжуулдаг сервер; (3) ui/index.html + model.js — fetch streaming-ээр event уншиж, SVG дээр state diagram зурдаг, DOM-гүй логикийг model.js-д тусгаарласан клиент. Өгөгдлийн урсгал: хэрэглэгч UI дээр тохиргоо (алгоритм, n, think/eat хугацаа, seed) оруулаад Эхлүүлэх дарахад → server.py нь build/dp-г тохирох аргументтайгаар эхлүүлж → dp процесс event бүрийг JSON мөр бүрээр stdout-д бичиж → server.py эдгээрийг HTTP streaming (chunked) байдлаар клиент рүү дамжуулж → model.js event-ийг боловсруулж snapshot үүсгэж → index.html SVG дээр бодит цагт шинэчилж зурдаг.

5. ХЭРЭГЖИЛТ

Ажлыг гүйцэтгэхэд дараах програм хангамж, хэл, сан ашиглагдсан:

- C (gnu11 стандарт), POSIX threads (pthread) — үндсэн логик (src/dp.c)
- GCC ≥ 9 compiler, -Wall -Wextra -O2 тохиргоотой
- Python 3 (стандарт сан) — localhost UI сервер (http.server)
- Vanilla JavaScript, SVG — интерактив state-diagram UI, гадны framework ашиглаагүй
- ThreadSanitizer (-fsanitize=thread) — race condition илрүүлэгч
- valgrind (helgrind, --leak-check=full) — race condition болон санах ойн алдагдал шалгагч
- Node.js — UI логикийн unit тест (tests/test_model.js)
- GNU Make — build, test автоматжуулалт
- Claude (Anthropic) — код боловсруулалт, тест бэлтгэл, тайлан найруулгад туслах зэвсэг болгон ашигласан [4].

Гол шийдлийн хувьд monitor хувилбарт pthread_mutex_t mon ба pthread_cond_t self_cv[n] ашиглагдсан. pickup_forks(i) функц philosopher i-г HUNGRY болгож test(i)-г дуудаад, хоёр хөрш нь EATING биш л бол EATING болгож, эс бөгөөс pthread_cond_wait-аар хүлээлгэдэг. Naive хувилбарт philosopher бүр pthread_mutex_t fork_mx[n]-ийн зүүн элементийг эхлээд (амжилттай болтол trylock-ыг 1мс тутам давтдаг grab_fork() функцээр) авч, --grab-delay мс хүлээгээд баруун fork-оо адил аргаар (trylock давтан) авахыг оролддог бөгөөд энэ хугацаанд зүүн fork-оо огт суллахгүй. --grab-delay параметр нь яг энэ "бүгд зүүн forkоо авсны дараа, хэн ч баруунаа оролдоогүй байгаа" synchronization цонхыг зориуд өргөсгөж, бүх philosopher зэрэг зүүн fork-оо авахад хангалттай болгодог тул deadlock-ыг тогтвортой давтагдахаар үүсгэдэг (доор 9-р хэсэгт --grab-delay=0 үеийн эсрэг жишээг үзүүлэв).

6. ТУРШИЛТ БА UI ДҮРСЛЭЛ

State-diagram UI нь philosopher бүрийн thinking/hungry/eating төлвийг, fork-ийн эзэмшилийг бодит цагт SVG дээр дүрсэлдэг бөгөөд хэрэглэгч дараах параметрүүдийг чөлөөтэй өөрчилж дахин ажиллуулах боломжтой: алгоритм (monitor/naive), philosopher-ийн тоо (n=2–64), бодох/идэх хугацааны муж (min/max, мс), ажиллах нийт хугацаа, санамсаргүй дараалал гаргах seed, болон naive хувилбарт зориулсан fork авах завсар (grab-delay). Юуг ажиглах вэ гэвэл: (1) philosopher бүрийн төлөв шилжилт (өнгөөр ялгагдана: цэнхэр=thinking, шар-улбар=hungry, ногоон=eating), (2) fork (шугам) бүрийн эзэмшил ба байрлал, (3) зэргэлдээх хоёр philosopher хэзээ ч зэрэг eating болохгүй байх, (4) статистикийн хүснэгтэд philosopher бүрийн хоолны тоо, дундаж/дээд хүлээлтийн хугацаа, (5) naive хувилбарт бүх philosopher hungry төлөвт царцаж,

улаан хүрээ, "DEADLOCK" баннер гарах эсэх. Нэмэлтээр ажиллагаа дууссаны дараа event бүрийг replay slider-ээр ухрааж харах боломжтой тул UI нь зөвхөн чимэглэл бус, ажиллагааг удирдах бодит интерфэйс болсон.

7. ТУРШИЛТЫН АРГА

Туршилт Linux орчинд (GCC -std=gnu11 -Wall -Wextra -O2 -pthread) явагдсан. Инвариант шалгалтад tests/check_log.py ашиглагдсан бөгөөд энэ скрипт JSON Lines гаралтаас хугацааны дараалал, төлөв шилжилтийн хууль ёсны байдал, mutual exclusion, fork-ийн цорын ганц эзэмшил, eating үеийн хоёр fork-ийн эзэмшилийг автоматаар шалгадаг. Нэмэлтээр ThreadSanitizer болон valgrind helgrind/leak-check ашиглан race condition, санах ойн алдагдлыг шалгасан. Орчин бүрт хэрэглэсэн өгөгдөл, хүлээгдэж буй үр дүнг доорх хүснэгтэд нэгтгэв.

Хүснэгт 7.1 — Туршилтын тохиргоонууд

№	Тохиргоо	Зорилго
1	monitor, n=5, think/eat=10-40мс, 4с, seed=1	Starvation ажиглагдаж байгаа эсэхийг шалгах (баталгаажуулалт биш)
2	naive, n=5, think=0, eat=300-600мс, grab-delay=300мс, seed=1	Deadlock детерминистээр давтагдах эсэхийг шалгах
3	monitor, n=2 (доод хязгаар), seed=2	Захын тохиолдол
4	monitor, n=64 (дээд хязгаар), seed=3	Захын тохиолдол
5	буруу аргумент (n=1, think-min>think-max)	Алдааны мэдээллийн зөв байдал

8. ҮР ДҮН

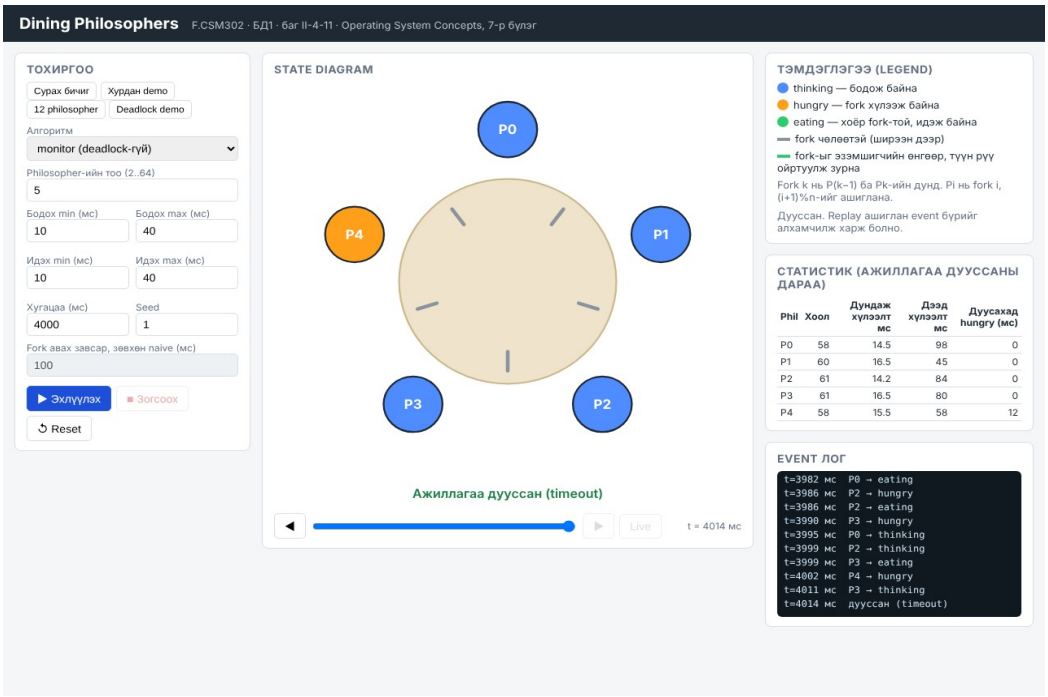
Автомат шалгалт (make test) дараах 16 тестийг амжилттай (OK) дүгнэсэн: compile, model.js unit тест (node tests/test_model.js шууд ажиллуулахад 22 passed, 0 failed), monitor инвариант (бүгд ≥3 удаа идсэн), naive deadlock (гарах код 3), n=2 болон n=64 захын тохиолдол, буруу аргументын алдааны мэдээлэл, UI серверийн health/index/run/буруу-параметр болон буруу бүтэцтэй тоон утга (жишээ нь "n=--5") шалгалт, ThreadSanitizer race condition monitor БА naive хоёуланд нь илрээгүй. Нэмэлт (make test-д орооогүй, гар аргаар `make memcheck`-ээр ажиллуулдаг) valgrind helgrind ба --leak-check=full команд — monitor, naive тус бүрд (нийт дөрвөн valgrind команд) — бүгд алдаагүй, хоосон гаралттай дууссан (туршилтын явцад олдсон нэг жинхэнэ race condition-ийг доор 9-р хэсэгт тайлбарлав).

Хүснэгт 8.1 — Monitor алгоритмын тоон үр дүн (n=5, think/eat=10-40мс, 4с, seed=1)

Philosopher	Хоолны тоо	Дундаж хүлээлт (мс)	Дээд хүлээлт (мс)
P0	58	14.5	98
P1	60	16.5	45

Philosopher	Хоолны тоо	Дундаж хүлээлт (мс)	Дээд хүлээлт (мс)
P2	61	14.2	84
P3	61	16.5	80
P4	58	15.5	58

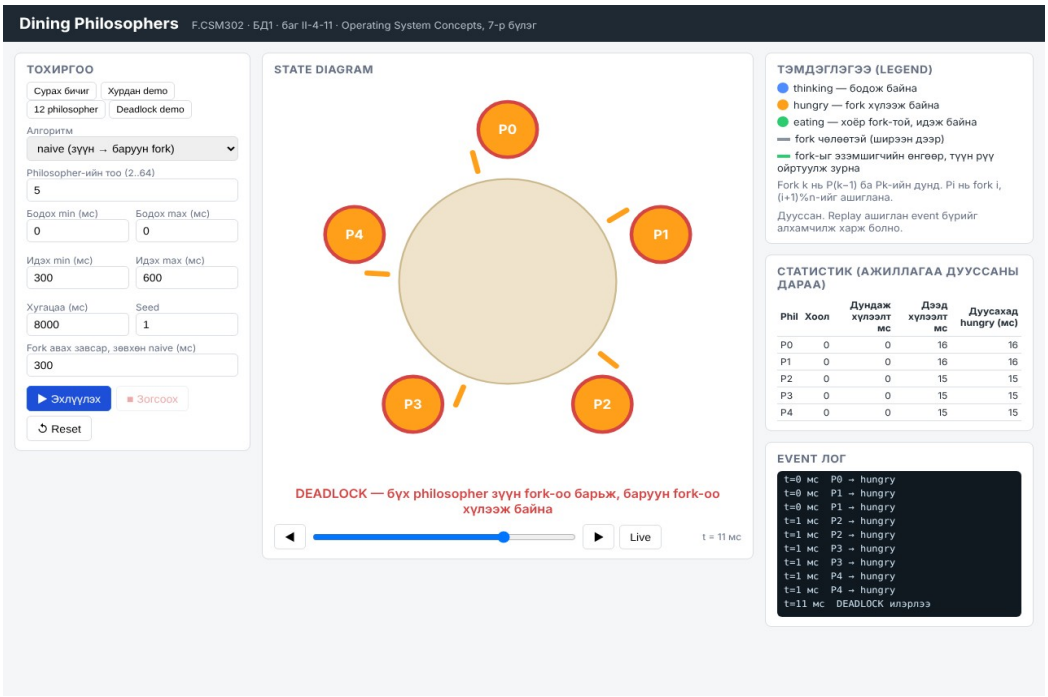
Хоолны тооны дундаж 59.6, стандарт хазайлт ≈1.36 (≈2.3%) — энэ ганц ажиллагааны хэмжилт нь monitor алгоритм энэ тохиргоонд philosopher бүрт ойролцоогоор тэнцүү тооны хоол олгосныг ажиглуулж байна; --seed нь зөвхөн рандом дугаар үүсгэгчийн (think/eat хугацааны) дараалыг детерминистээр баталгаажуулдаг бөгөөд thread-үүдийн БОДИТ ажиллах цаг хугацааны (OS scheduling) дараалыг баталгаажуулдаггүй тул ижил seed-тэй ч гэсэн ажиллагаа бүрт хоолны тоо бага зэрэг өөрчлөгдөж болно (энэ хэвлэлд [58,60,61,61,58], өөр ажиллагаанд [58,63,63,60,57] гэх мэт) — энэ нь fairness-ийн статистик баталгаа биш, нэг төлөөлөх жишээ гэдгийг тэмдэглэх нь зүйтэй.



Зураг 8.1 — Monitor алгоритм, n=5, think/eat=10-40мс, 4с, seed=1 (Хүснэгт 8.1-тэй ижил тохиргоо, зурагны screenshot нь яг тус хэмжилтээс шууд авсан), ажиллагаа дууссаны дараах state-diagram UI ба статистикийн хүснэгт (шинэ "Дуусахад hungry" багана: P4 ганцаараа яг 4000мс дуусах мөчид hungry төлөвт байсан тул дээд хүлээлт нь (58мс) үүгээр тодорхойлогдсон)

Naive хувилбарыг think=0, eat=300-600мс, grab-delay=300мс тохиргоогоор ажиллуулахад бүх 5 philosopher t=0-1мс орчимд бараг нэгэн зэрэг зүүн fork-оо авч, дараа нь баруун fork-оо (trylock давтан оролдож) чадахгүй, зүүн fork-оо суллахгүйгээр хүлээсээр үлдсэн тул (meals=[0,0,0,0,0]) нэг ч хоол идэж амжаагүй, систем DEADLOCK төлөвт шилжсэн нь дор харуулав. Доорх screenshot нь UI-ийн replay slider-ийг яг "DEADLOCK илэрлээ" event (t=11мс) дээр нь байрлуулж (дараагийн fork-release cleanup event-үүдээс ӨМНӨ) авсан тул бүх fork орон сууриа орхиогүй, эзэмшигчийн

өнгөөр (hungry-гийн улбар шар) зурагдсан харагдана; statistics хүснэгт нь (ажиллагаа бүхэлдээ дуусмагц нэг удаа бичигддэг тул slider-ийн байрлалаас үл хамаарч) эцсийн "Дуусахад hungry" утгыг харуулж байна — [16,16,15,15,15]мс, бүгд deadlock-ийн t=11мс-тэй ойролцоо, баннертай бүрэн нийцнэ (Хавсралт В-ийн бодит лог, 9-р хэсэг).



Зураг 8.2 — Naive алгоритм дахь детерминист deadlock, яг илрэх мөчид t=11мс (бүх philosopher hungry, зүүн fork-оо барьсан, улаан хүрээтэй)

9. ХЭЛЭЛЦҮҮЛЭГ

Naive хувилбарыг зориудаар нэмж, deadlock-ыг зөвхөн онолын тайлбараар бус, бодит хэмжилтээр (Хүснэгт 7.1-ийн 2-р мөр, Зураг 8.2) харуулсан. Хэмжсэн утга (meals=[0,0,0,0,0], 3/3 туршилтад deadlock илэрсэн) нь онолын урьдчилсан таамаглалтай (circular wait → deadlock) нийцэж байна. Deadlock илрэх t-утга (Зураг 8.2-т t=11мс, Хавсралт В-ийн жишээ логт t=10мс) ажиллагаа бүрт OS-ийн thread scheduling-ийн улмаас хэдхэн мс-ээр хэлбэлздэг ч илрэлтийн зарчим тогтвортой давтагдсан.

Анхаарал татахуйц нэг нэмэлт туршилт: --grab-delay-г 0 болговол (think=0 хэвээр, зөвхөн зүүн ба баруун fork авах хоорондох хиймэл хоцролтыг арилгавал) deadlock-ыг бэхжүүлэхгүй, харин эсрэгээр 3 удаагийн давтан туршилтад (seed=1,2,3) нэг ч удаа бүрэн deadlock үзүүлээгүй: харин philosopher-үүдийн хооронд ноцтой тэгш бус хуваарилалт (starvation) ажиглагдсан — жишээ нь нэг ажиллагаанд (seed=2, n=5, think=0, eat=300-600мс, -d 4000) meals=[9,0,10,0,0], өөрөөр хэлбэл P1, P3, P4 нэг ч удаа идэж чадаагүй бөгөөд тэдний "дуусахад хэвээр hungry байсан" хугацаа ≈4010мс (бараг бүх ажиллагааны туршид) хүрсэн нь dp.c-ийн шинэчилсэн final_hungry_wait_ms

талбараар шууд хэмжигдсэн. Анхаарал татахуйц нэг зүйл: ялагч philosopher-үүд (жишээ нь seed=1,2,3 бүгдэд id=0 ба өөр нэг тэгш/хөрш бус id) санамсаргүй бус, ажиллагаа бүрт ижил хэв маягаар давтагдав — энэ нь "OS-ийн санамсаргүй sched товлолтоор" гэх таамаглал буруу байж болзошгүйг, харин thread-ийн ЭХЛЭХ ДАРААЛАЛ (pthread_create-ийн дараалал) эсвэл glibc-ийн thread/fork-ийн анхны scheduling-тэй холбоотой ДЕТЕРМИНИСТ шинж чанар байж магадгүйг харуулж байна. Энэ ажлын хүрээнд яг шалтгааныг нь нарийвчлан судлаагүй тул зөвхөн баримт болгон ажиглалтыг тэмдэглэв — цаашдын судалгаанд зохистой сэдэв. Хэдий тийм ч --grab-delay параметрийн зорилго нь бүх philosopher зэрэг зүүн fork-оо авах synchronization цонхыг зориуд өргөсгөж, deadlock-ыг тогтвортой болгоход оршдог гэдгийг энэ туршилт тоон баримтаар харуулж байна (3 ажиллагаа бол ердөө л нэг жишиг тохиолдол, статистик баталгаа биш).

Туршилтын явцад анхны хувилбарт нэг бодит race condition олдсон: main()-д deadlock илэрсэн үед хэвлэдэг {"type":"deadlock",...} мөрийг бусад philosopher thread-үүд (pthread_join хараахан дуудагдаагүй тул) ҮРГЭЛЖИЛСЭЭР ажиллаж байхад, event лог бичдэг ev_lock mutex-ийг АШИГЛАХГҮЙГЭЭР хэвлэж байсан нь log_state()-ийн хамгаалалттай зэрэгцэн ажиллаж болзошгүй stdout бичилтийн зөрчил (race) үүсгэж байв (helgrind-ээр `make memcheck`-ийн дөрвөн командын (monitor/naive × helgrind/leak-check) зарим давталтад "Possible data race" гэж илэрсэн). Мөн main()-ы эхний {"type":"config",...} мөрийг хэвлэх нь онолын хувьд pthread_create()-аас өмнө, дан ганц thread-ээр явагддаг тул өрсөлдөөнгүй байх ёстой ч зарим динамик шинжилгээний хэрэгсэл glibc-ийн шинэ дотоод stdio буфертэй холбоотойгоор дэмий сэрэмжлүүлэх эрсдэлтэй. Иймд хоёр хэвлэлтийг ч ev_lock-оор хамгаалж засав (src/dp.c), үүний дараа `make memcheck`-ийн дөрвөн valgrind команд давтан туршилтад бүгд цэвэр дууссан. Мөн энэ явцад Makefile-ийн memcheck зорилтод нэг нэмэлт алдаа олдсон: naive тохиргоо нь санаатайгаар deadlock-д орж ./build/dp exit code 3-тай гардаг тул valgrind уг кодыг дамжуулж, make үүнийг "алдаа" гэж буруу тайлбарладаг байв — valgrind-д --error-exitcode=99 нэмж, зөвхөн valgrind ӨӨРӨӨ race/leak олсон тохиолдолд л алдаа гэж тооцохоор засав.

Хязгаарлалтын хувьд: UI сервер нь зэрэгцээ ажиллагааг 4-өөр хязгаарласан (BoundedSemaphore) тул олон хэрэглэгч нэгэн зэрэг Start дарвал сүүлчийнх нь 429 алдаа авч болзошгүй; naive алгоритмын deadlock нь зөвхөн тогтмол (санамсаргүй бус) бодох/идэх хугацаанд детерминистээр давтагддаг бөгөөд санамсаргүй think-хугацаатай тохиолдолд давтамжийг статистик байдлаар хэмжээгүй болно; мөн туршилт зөвхөн Linux орчинд хийгдсэн тул Windows/macOS дээрх pthread хэрэгжилтийн зан төлөвийг тусад нь шалгаагүй болно.

10. ДҮГНЭЛТ

Энэхүү ажлаар Dining-Philosophers Problem-ийг C/pthread дээр хоёр алгоритмаар (monitor, naive) хэрэгжүүлж, инвариант-суурьтай болон тоон шалгалтаар monitor хувилбарын deadlock-гүй ажиллагааг, naive хувилбарын тогтвортой давтагддаг deadlock-ыг харуулсан. Шалгасан тохиргоонуудад (n=2, 5, 64) инвариант зөрчигдөөгүй, ThreadSanitizer, valgrind тухайн ажиллагаанд race condition, санах ойн алдагдал илрүүлээгүй, 16/16 автомат шалгалт амжилттай дууссан нь хэрэгжилтийн зөв ажиллагааг нотолж байна — гэхдээ эдгээр нь шалгалт (testing), албан ёсны баталгаажуулалт (formal verification) биш тул race condition байхгүй гэдгийг 100% баталгаажуулдаггүй, зөвхөн тухайн ажиллагаанд илрээгүйг л харуулдаг. Эндээс monitor-ийн сурах бичигт заасан онолын шинж чанар (deadlock-гүй, mutual exclusion хангагдсан; starvation онолын хувьд боломжтой ч манай туршилтад ажиглагдаагүй) практикт тохирч байгааг дүгнэж болно.

АШИГЛАСАН МАТЕРИАЛ

[1] Silberschatz, A., Galvin, P. B., Gagne, G. "Operating System Concepts", 10th Edition, Wiley, 2018 — 6-р бүлэг "Synchronization Tools" (Monitor-ийн ерөнхий ойлголт), 7-р бүлэг "Synchronization Examples" (7.1.3 The Dining-Philosophers Problem, x. 293-294, Figure 7.7 — monitor шийдэл, starvation-ийн тухай тэмдэглэл), "Programming Projects — Project 3: The Dining-Philosophers Problem" (x. P-39), болон 8-р бүлэг "Deadlocks" (deadlock-ийн дөрвөн шаардлагатай нөхцөл).

[2] Linux man-pages project / The Open Group. "pthread_cond_wait, pthread_cond_timedwait — wait on a condition variable" — POSIX Threads Programming. https://man7.org/linux/man-pages/man3/pthread_cond_wait.3p.html

[3] F.CSM302 "Бие даалт 1" заавар (OS_BD1_Guide.pdf) — хуваарилалтын код II-4-11, сэдвийн шаардлага.

[4] Claude (Anthropic) — код бичих, тест бэлтгэх, тайлан боловсруулахад ашигласан бөгөөд бүх үр дүнг make test, ThreadSanitizer, valgrind-аар бодитоор баталгаажуулсан.

ХАВСРАЛТ

Хавсралт А — Ажлын төлөвлөгөө

Талбар	Мэдээлэл
Хуваарилалтын код	II-4-11
Төсөл	The Dining-Philosophers Problem
Repository	https://github.com/Selenge555/-FCSA_302_II-4-11-dining-philosophers

Талбар	Мэдээлэл
Хэл/сан/орчин	C (gnu11, pthread), Python 3 (http.server), Vanilla JS/SVG, GCC, GNU Make, valgrind, ThreadSanitizer
Үндсэн дүрслэл	State diagram (интерактив SVG, бодит цагийн philosopher/fork төлөв)

Гишүүн	Хариуцах ажил	Commit/file
Г. Сэлэнгэ (B242270106)	Үндсэн архитектур, monitor/naive алгоритм	src/dp.c, Makefile
Ч. Гантогтох (B242270005)	UI сервер, /run endpoint, параметр шалгалт	ui/server.py
Э. Энхсаруул (B232270145)	Frontend UI, бодит цагийн SVG дүрслэл	ui/index.html, ui/model.js
Э. Төгөлдөр (B241940019)	Автомат тест, ThreadSanitizer/valgrind шалгалт	tests/run_tests.sh, README.md

№	Тест — Оролт/нөхцөл	Хүлээгдсэн үр дүн
1	monitor, n=5, think/eat=10-40мс, 4с, seed=1	Starvation ажиглагдаагүй (бүгд ≥3 удаа идэх), инвариант зөрчигдөхгүй
2	naive, n=5, think=0, eat=300-600мс, grab-delay=300мс	Детерминист deadlock, гарах код 3
3	monitor, n=2 (доод хязгаар)	Инвариант зөрчигдөхгүй
4	monitor, n=64 (дээд хязгаар)	Инвариант зөрчигдөхгүй
5	буруу аргумент (n=1, think-min>think-max)	Гарах код 2, ойлгомжтой алдааны мессеж
6	ThreadSanitizer (-fsanitize=thread), monitor ба naive	WARNING race condition гарахгүй

Хавсралт Б — Автомат шалгалтын бүрэн гаралт (make test, bash tests/run_tests.sh-ийн шууд, засваргүй stdout)

```
== 1) build ==========================================
  OK  build/dp compile хийгдлээ
== 2) model.js unit тест ============================
  OK  test_model.js бүх шалгалт давсан
== 3) интеграц -- monitor, инвариант ба min-meals ======
  OK  monitor: инвариант хангасан, бүгд дор хаяж 3 удаа идсэн
== 4) интеграц -- naive, deadlock =====================
  OK  naive: гарах код 3 (deadlock)
  OK  naive: инвариант хангасан, deadlock илэрсэн
== 5) захын тохиолдол -- n=2, n=64 ====================
  OK  n=2 (хамгийн бага) инвариант хангасан
  OK  n=64 (хамгийн их) инвариант хангасан
== 6) буруу аргумент зөв мэдээлдэг эсэх ================
  OK  буруу аргументад гарах код 2, ойлгомжтой алдааны мессежтэй
== 7) UI сервер =======================================
  OK  сервер /health хариулж байна
  OK  сервер / -ээс index.html хариулж байна
  OK  сервер /run эхний event нь config
  OK  сервер буруу параметрт 400/errors буцаасан
  OK  сервер 'n=--5' маягийн буруу тоог унахгүйгээр 400 буцаасан
  OK  сервер буруу оролтын дараа ч амьд хэвээр
== 8) ThreadSanitizer (боломжтой бол, monitor ба naive хоёулаа) ==
  OK  ThreadSanitizer (monitor): race condition илрээгүй
  OK  ThreadSanitizer (naive): race condition илрээгүй
```

```
============================================================
Нийт: 16 OK, 0 FAIL
```

Хавсралт В — naive алгоритмын deadlock-ийн бодит event лог (JSON Lines, ./build/dp-ийн шууд гаралт)

Доорх лог нь Хүснэгт 7.1-ийн 2-р мөрийн тохиргоотой (naive, n=5, think=0, eat=300-600мс, grab-delay=300мс, seed=1) ./build/dp-г шууд ажиллуулж авсан бодит, засваргүй гаралт (ev_lock-ын засвар, final_hungry_wait_ms талбар нэмэгдсэний дараах хувилбар). Зураг 8.2 нь UI-аар тусад нь авсан өөр нэг ажиллагаа тул t-утгууд (илрэлт: Хавсралт В-ийн лог t=10мс, Зураг 8.2 t=11мс) бага зэрэг ялгаатай — энэ нь OS-ийн thread scheduling-ийн нөлөөгөөр тайлбарлагдана; гэхдээ үр дүн (бүрэн deadlock, meals=[0,0,0,0,0], бүх philosopher дуусах мөчдөө ХЭВЭЭР hungry) хоёр ажиллагаанд ч ижил, тогтвортой давтагдсан. Анхаарах нь: deadlock илэрсний дараах сүүлийн таван "state" мөрөнд fork-ийн эзэмшил -1 болж харагдах нь philosopher дахин оролдож амжсан хэрэг БИШ — энэ бол main() stop_flag тавьсны дараа грэйсфул (graceful) унтрах зорилгоор trylock-ын давталтаас гарч, эзэмшсэн fork-оо суллах цэвэрлэгээ (cleanup) бөгөөд state нь HUNGRY хэвээр, нэг ч philosopher хэзээ ч THINKING/EATING рүү шилжээгүй.


```
{"type":"config","algorithm":"naive","n":5,"think_min":0,"think_max":0,"eat_min":300,"eat_max
":600,"duration_ms":8000,"seed":1,"grab_delay_ms":300}
{"type":"state","t":0,"id":0,"state":"thinking","forks":[-1,-1,-1,-1,-1]}
{"type":"state","t":0,"id":1,"state":"thinking","forks":[-1,-1,-1,-1,-1]}
{"type":"state","t":0,"id":2,"state":"thinking","forks":[-1,-1,-1,-1,-1]}
{"type":"state","t":0,"id":3,"state":"thinking","forks":[-1,-1,-1,-1,-1]}
{"type":"state","t":0,"id":4,"state":"thinking","forks":[-1,-1,-1,-1,-1]}
{"type":"state","t":0,"id":1,"state":"hungry","forks":[-1,-1,-1,-1,-1]}
{"type":"state","t":0,"id":1,"state":"hungry","forks":[-1,1,-1,-1,-1]}
{"type":"state","t":0,"id":2,"state":"hungry","forks":[-1,1,-1,-1,-1]}
{"type":"state","t":0,"id":2,"state":"hungry","forks":[-1,1,2,-1,-1]}
{"type":"state","t":0,"id":3,"state":"hungry","forks":[-1,1,2,-1,-1]}
{"type":"state","t":0,"id":3,"state":"hungry","forks":[-1,1,2,3,-1]}
{"type":"state","t":0,"id":4,"state":"hungry","forks":[-1,1,2,3,-1]}
{"type":"state","t":0,"id":4,"state":"hungry","forks":[-1,1,2,3,4]}
{"type":"state","t":0,"id":0,"state":"hungry","forks":[-1,1,2,3,4]}
{"type":"state","t":0,"id":0,"state":"hungry","forks":[0,1,2,3,4]}
{"type":"deadlock","t":10}
{"type":"state","t":10,"id":3,"state":"hungry","forks":[0,1,2,-1,4]}
{"type":"state","t":15,"id":0,"state":"hungry","forks":[-1,1,2,-1,4]}
{"type":"state","t":15,"id":4,"state":"hungry","forks":[-1,1,2,-1,-1]}
{"type":"state","t":15,"id":2,"state":"hungry","forks":[-1,1,-1,-1,-1]}
{"type":"state","t":15,"id":1,"state":"hungry","forks":[-1,-1,-1,-1,-1]}
{"type":"end","t":15,"reason":"deadlock","meals":[0,0,0,0,0],"avg_wait_ms":
[0.0,0.0,0.0,0.0,0.0],"max_wait_ms":[15,15,15,15,15],"final_hungry_wait_ms":[15,15,15,15,15]}
```